

Memoria Técnica — MusicPlayer

1. Introducción

1.1 Descripción del proyecto

MusicPlayer es una plataforma web de reproducción y gestión de música desarrollada como Trabajo de Fin de Grado (TFG). Permite a los usuarios escuchar música, gestionar listas de reproducción, seguir artistas y descubrir nuevo contenido. El sistema soporta tres roles diferenciados: administrador, editor y oyente.

1.2 Objetivos

- Desarrollar una aplicación web completa (frontend + backend) con arquitectura cliente-servidor.
- Implementar un sistema de autenticación basado en tokens JWT con control de acceso por roles.
- Crear una interfaz de usuario moderna e intuitiva inspirada en plataformas de streaming actuales.
- Integrar una API externa para mostrar letras de canciones en tiempo real.
- Documentar el sistema completo para facilitar su mantenimiento y evolución.

1.3 Alcance

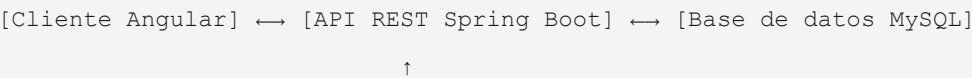
El sistema cubre:

- Registro, autenticación y gestión de usuarios.
- Catálogo de artistas, álbumes, géneros y canciones con operaciones CRUD.
- Sistema de listas de reproducción y gestión de canciones favoritas.
- Historial de reproducciones por usuario.
- Dashboards diferenciados según el rol del usuario.
- Obtención de letras de canciones mediante API externa.

2. Arquitectura del sistema

2.1 Visión general

La aplicación sigue una arquitectura de tres capas desacopladas:



[API externa: lyrics.ovh]

El frontend Angular consume la API REST del backend mediante peticiones HTTP autenticadas con JWT. El backend gestiona la lógica de negocio, la persistencia y la seguridad. Para las letras de canciones, el frontend consulta directamente la API pública `lyrics.ovh`.

2.2 Framework base: JHipster 9.0.0

Se eligió JHipster como generador de código base por los siguientes motivos:

- Genera automáticamente la estructura inicial del proyecto con las mejores prácticas de Spring Boot y Angular.
- Incluye configuración de seguridad JWT preintegrada.
- Proporciona herramientas de migración de base de datos (Liquibase) y mapeo de entidades (MapStruct).
- Reduce el tiempo dedicado a la configuración inicial y permite centrarse en la lógica de negocio y el diseño de interfaz.

2.3 Capa frontend

Construida con **Angular 21** en modo standalone. Se comunica con el backend mediante `HttpClient` y gestiona el estado de sesión a través de JWT almacenado en `localStorage`.

Estructura principal:

```
src/main/webapp/app/
├─ app.config.ts      - Configuración de providers (HttpClient, Router, i18n)
├─ app.routes.ts      - Definición de rutas con guards
├─ core/              - Servicios transversales (auth, interceptores, config)
├─ layouts/           - Componentes de maquetación (navbar, sidebar, player-bar)
├─ home/              - Dashboards por rol (admin, editor, usuario)
├─ entities/          - Módulos CRUD de entidades (song, album, artist, etc.)
├─ account/           - Gestión de cuenta de usuario
├─ login/             - Página de autenticación
└─ shared/            - Componentes y utilidades compartidas
```

2.4 Capa backend

Construida con **Spring Boot 4.0.3** y Java. Expone una API REST protegida por JWT gestionada mediante Spring Security con el módulo OAuth2 Resource Server. Organizada en paquetes con separación clara de responsabilidades:

```
com.musicplayer
├─ config/            - 18 clases de configuración (Security, DB, Cache, Mail, etc.)
```

└─ domain/	– 11 entidades JPA + enumeración AlbumType
└─ repository/	– 12 interfaces Spring Data JPA
└─ service/	– 9 interfaces de servicio + 8 implementaciones
└─ service.dto/	– 11 DTOs de transferencia
└─ service.mapper/	– 10 mappers MapStruct (entidad ↔ DTO)
└─ web.rest/	– 13 controladores REST
└─ web.rest.errors/	– Manejo global de excepciones
└─ security/	– Utilidades y configuración de seguridad JWT

El acceso a cada endpoint está controlado por roles mediante anotaciones Spring Security (`@PreAuthorize`). Las sesiones son completamente **stateless**: cada petición se autentica de forma independiente mediante el token JWT.

2.5 Capa de datos

Base de datos **MySQL 8** en producción; **H2 en memoria** para desarrollo. Las migraciones de esquema se gestionan exclusivamente con **Liquibase** (`spring.jpa.hibernate.ddl-auto: none`), garantizando trazabilidad y reproducibilidad. El esquema incluye 10 changelogs versionados que crean las tablas de negocio, relaciones y roles personalizados de forma incremental.

3. Capa backend

3.1 API REST

El backend expone **13 controladores REST** bajo la ruta base `/api` . Todos los endpoints de negocio requieren autenticación mediante `Authorization: Bearer <token>` .

Recurso	Ruta base	Operaciones
Canciones	<code>/api/songs</code>	GET (paginado), GET /{id}, POST, PUT, DELETE
Álbumes	<code>/api/albums</code>	GET (paginado), GET /{id}, POST, PUT, DELETE
Artistas	<code>/api/artists</code>	GET (paginado), GET /{id}, POST, PUT, DELETE
Géneros	<code>/api/genres</code>	GET (paginado), GET /{id}, POST, PUT, DELETE
Playlists	<code>/api/playlists</code>	GET (paginado), GET /{id}, POST, PUT, DELETE
Canciones de playlist	<code>/api/playlist-songs</code>	GET, POST, PUT, DELETE
Reproducciones	<code>/api/plays</code>	GET, POST, DELETE
Favoritos	<code>/api/likes</code>	GET, POST, DELETE

Recurso	Ruta base	Operaciones
Usuarios (admin)	/api/admin/users	GET, POST, PUT, DELETE
Cuenta	/api/account	GET, POST (actualizar perfil, cambiar contraseña)
Autenticación	/api/authenticate	POST (login), GET (verificar sesión activa)
Registro	/api/register	POST
Activación	/api/activate	GET (?key=...)

Las listas paginadas devuelven la cabecera `X-Total-Count` con el total de registros, compatible con el componente de paginación JHipster del frontend. Las respuestas siguen el estándar HTTP (201 Created en POST, 200 OK en GET/PUT, 204 No Content en DELETE).

3.2 Modelo de dominio

El modelo de datos cuenta con **10 entidades JPA** de negocio:

Entidad	Campos principales	Relaciones
Song	title, duration, fileUrl, coverImage, lyrics, releaseDate	ManyToOne → Album, Genre; ManyToMany ↔ Artist
Album	title, coverImage, releaseDate, albumType	ManyToOne → Artist, Genre
Artist	name, bio, image, country, verified	ManyToMany ↔ Song
Genre	name (único)	—
Playlist	name, description, isPublic, coverImage	ManyToOne → User
PlaylistSong	position, addedAt	ManyToOne → Playlist, Song
Play	playedAt, durationListened	ManyToOne → User, Song
Like	createdAt	ManyToOne → User, Song
User	login, email, firstName, lastName, activated, langKey	ManyToMany ↔ Authority
Authority	name (PK)	—

Todas las entidades de negocio extienden `AbstractAuditingEntity`, que añade automáticamente los campos de auditoría `createdBy`, `createdDate`, `lastModifiedBy` y `lastModifiedDate` mediante Spring Data JPA Auditing.

La enumeración `AlbumType` permite clasificar los álbumes: `ALBUM`, `SINGLE`, `EP`, `PODCAST_SERIES`.

La relación ManyToMany entre `Song` y `Artist` usa una interfaz especializada

`SongRepositoryWithBagRelationships` que carga los artistas mediante consultas separadas para evitar el problema de producto cartesiano de Hibernate en colecciones *bag*.

3.3 Capa de servicio y patrón DTO

Se sigue el patrón **Controlador** → **Servicio** → **Repositorio** con DTOs para desacoplar la capa de presentación del modelo de dominio:

1. Los controladores REST reciben y devuelven **DTOs** (11 clases en `service.dto`).
2. Los **servicios** (`service.impl`) contienen la lógica de negocio y orquestan repositorios.
3. Los **mappers MapStruct** (`service.mapper`) convierten automáticamente entre entidades JPA y DTOs generando el código en tiempo de compilación, sin reflexión en tiempo de ejecución.

Flujo de creación de canción como ejemplo:

```
POST /api/songs (SongDTO)
  → SongResource
    → SongService.save(SongDTO)
      → SongMapper.toEntity(SongDTO)
        → SongRepository.save(Song)
          → SongMapper.toDto(Song)
    ← ResponseEntity<SongDTO> 201 Created
```

Los servicios implementan interfaces (`SongService` , `AlbumService` , `PlaylistService` , etc.) siguiendo el principio de inversión de dependencias, lo que facilita su sustitución y prueba unitaria.

3.4 Persistencia y migraciones Liquibase

Las migraciones están versionadas en `src/main/resources/config/liquibase/`:

Changeset	Contenido
0000000000000000_initial_schema.xml	Tablas de usuario JHipster (<code>jhi_user</code> , <code>jhi_authority</code> , <code>jhi_user_authority</code>)
20260410183941_added_entity_Genre.xml	Tabla <code>genre</code>
20260410184041_added_entity_Artist.xml	Tabla <code>artist</code>
20260410184141_added_entity_Album.xml	Tabla <code>album</code> + FK a artist y genre; índices
20260410184241_added_entity_Song.xml	Tabla <code>song</code> + tabla de relación <code>rel_song__artists</code>

Changeset	Contenido
20260410184341_added_entity_Playlist.xml	Tabla <code>playlist</code> + FK a usuario
20260410184441_added_entity_PlaylistSong.xml	Tabla <code>playlist_song</code> con campo <code>position</code>
20260410184541_added_entity_Play.xml	Tabla <code>play</code> (historial de reproducciones)
20260410184641_added_entity_Like.xml	Tabla <code>jhi_like</code> (canciones favoritas)
20260501210000_add_editor_artist_authorities.xml	Inserta <code>ROLE_EDITOR</code> y <code>ROLE_ARTIST</code> en <code>jhi_authority</code>

Este enfoque permite desplegar la aplicación en cualquier entorno ejecutando automáticamente solo los changesets pendientes, sin riesgo de duplicar cambios.

3.5 Seguridad del backend

Spring Security gestiona la autenticación y autorización. Las clases principales son `SecurityConfiguration` y `SecurityJwtConfiguration`.

Generación y validación de tokens JWT:

- Biblioteca: **Nimbus JOSE+JWT** (integrada en Spring Security OAuth2 Resource Server)
- Algoritmo: **HS512** (HMAC con SHA-512)
- Validez del token: **1800 segundos** (30 minutos)
- El secreto se configura codificado en Base64 en la propiedad

```
jhipster.security.authentication.jwt.base64-secret
```

Flujo de autenticación:

1. Cliente envía `POST /api/authenticate` con `{username, password, rememberMe}`
2. `AuthenticateController` delega en Spring Security Authentication Manager
3. `DomainUserDetailsService` carga el usuario de la base de datos y verifica credenciales
4. Si válido, `JwtEncoder` genera el token firmado con HS512 → devuelve al cliente
5. El cliente incluye `Authorization: Bearer <token>` en peticiones posteriores
6. `SecurityConfiguration` configura el servidor de recursos OAuth2 que valida el token en cada petición

Control de acceso por endpoint:

```
// Ejemplo: solo ROLE_ADMIN puede crear usuarios
@PreAuthorize("hasAuthority(\"\" + AuthoritiesConstants.ADMIN + "\")")
public ResponseEntity<AdminUserDTO> createUser(...) { ... }
```

Roles disponibles (`AuthoritiesConstants.java`):

Constante	Valor	Acceso
ADMIN	ROLE_ADMIN	Gestión completa del sistema y usuarios
USER	ROLE_USER	Oyente: playlists, favoritos, historial
EDITOR	ROLE_EDITOR	Gestión del catálogo musical
ARTIST	ROLE_ARTIST	Gestión del propio contenido

`ROLE_EDITOR` y `ROLE_ARTIST` son roles personalizados añadidos al esquema JHipster base mediante una migración Liquibase dedicada.

3.6 Manejo de errores

El manejo global se centraliza en `ExceptionHandler` (`@ControllerAdvice`), que convierte las excepciones del dominio en respuestas HTTP estructuradas compatibles con **RFC 7807 Problem Details**:

Excepción	HTTP	Descripción
<code>BadRequestAlertException</code>	400	Datos inválidos (incluye nombre de entidad y campo)
<code>LoginAlreadyUsedException</code>	400	Login de usuario ya registrado
<code>EmailAlreadyUsedException</code>	400	Email ya registrado
<code>InvalidPasswordException</code>	400	Contraseña no cumple criterios mínimos
Token inválido / expirado	401	No autenticado
Sin permiso de rol	403	Acceso denegado
Recurso no encontrado	404	Entidad con id solicitado no existe

3.7 Servicio de correo

`MailService` gestiona el envío de emails mediante **Spring Mail** con plantillas **Thymeleaf**:

- **Correo de activación:** se envía al registrar un nuevo usuario; contiene un enlace único a `/api/activate?key=...` que activa la cuenta.
- **Correo de restablecimiento:** se envía al solicitar un reset de contraseña; incluye un token de un solo uso con expiración.

En el perfil de desarrollo la configuración del servidor SMTP apunta a localhost para evitar envíos reales durante el desarrollo y las pruebas.

3.8 Perfiles de despliegue

Perfil	Base de datos	Uso
dev	H2 en memoria	Desarrollo local; esquema recreado en cada arranque
prod	MySQL 8	Producción; datos persistentes entre reinicios
tls	—	Habilita HTTPS/TLS sobre cualquier perfil

Diferencias clave entre perfiles:

- **Dev:** consola H2 accesible en `/h2-console`, logs SQL de Hibernate activos, sin caché L2.
- **Prod:** pool de conexiones HikariCP, logs SQL desactivados, caché de segundo nivel activa, seguridad CORS estricta.

3.9 Monitorización y logging

- **Spring Boot Actuator** expone `/management/health`, `/management/info`, `/management/metrics` y `/management/liquibase`; protegidos para `ROLE_ADMIN`.
- **AOP Logging:** `LoggingAspect` intercepta todas las llamadas a métodos de los paquetes `repository`, `service` y `web.rest`, registrando entradas, salidas, tiempos y excepciones sin modificar el código de negocio.
- **Caché L2 de Hibernate:** configurada con Infinispan/EhCache para entidades de referencia frecuentemente leídas (géneros, autoridades).

3.10 Patrón de propiedad en la capa de servicio (Ownership Pattern)

Uno de los requisitos funcionales clave del proyecto es que cada usuario solo pueda crear y modificar contenido que le pertenezca: un artista no puede editar canciones de otro artista, y un oyente no puede acceder a las playlists privadas de otro usuario. Este requisito se implementa mediante un **patrón de propiedad** aplicado consistentemente en la capa de servicio, no en los controladores REST.

La decisión de centralizar la verificación de propiedad en los servicios (en lugar de los controladores) sigue el principio de responsabilidad única: los controladores se limitan a recibir peticiones HTTP y devolver respuestas, mientras que la lógica de negocio —incluyendo quién puede hacer qué— reside en los servicios.

3.10.1 Asignación automática de artista en `SongServiceImpl`

Cuando un usuario con perfil de artista crea una canción, no se le pide que indique explícitamente qué artista es: el sistema lo determina automáticamente a partir del usuario autenticado.


```

@Override
public SongDTO save(SongDTO songDTO) {
    Song song = songMapper.toEntity(songDTO);

    // Obtener login del usuario autenticado desde el contexto de seguridad
    String login = SecurityUtils.getCurrentUserLogin()
        .orElseThrow(() -> new RuntimeException("No user logged"));

    // Buscar el perfil de artista asociado a ese login
    Artist artist = artistRepository.findByUserLogin(login)
        .orElseThrow(() -> new RuntimeException("Artist not found"));

    // Asignar el artista automáticamente antes de persistir
    song.setArtist(artist);
    song = songRepository.save(song);

    return songMapper.toDto(song);
}

```

`SecurityUtils.getCurrentUserLogin()` extrae el nombre de usuario del `SecurityContext` de Spring Security, que se puebla automáticamente en cada petición autenticada mediante el filtro JWT. El resultado es un `Optional<String>` que falla explícitamente si no hay sesión activa.

El método `ArtistRepository.findByUserLogin(login)` ejecuta una consulta JPQL personalizada:

```

@Query("SELECT a FROM Artist a WHERE a.user.login = :login")
Optional<Artist> findByUserLogin(@Param("login") String login);

```

Esta consulta realiza un `JOIN` implícito entre la entidad `Artist` y su relación `@ManyToOne` con `User`, filtrando por el campo `login` del usuario vinculado. El resultado es un `Optional<Artist>` que permite manejar elegantemente el caso de un usuario sin perfil de artista.

3.10.2 Ownership en AlbumServiceImpl con BadRequestAlertException

`AlbumServiceImpl` aplica el mismo patrón pero usa `BadRequestAlertException` en lugar de `RuntimeException`, lo que produce una respuesta HTTP 400 estructurada con campos de diagnóstico:

```

@Override
public AlbumDTO save(AlbumDTO albumDTO) {
    Album album = albumMapper.toEntity(albumDTO);

```

```
String login = SecurityUtils.getCurrentUserLogin()
    .orElseThrow(() ->
        new BadRequestAlertException("Usuario no autenticado", "album", "usernotfound"));

Artist artist = artistRepository.findByUserLogin(login)
    .orElseThrow(() ->
        new BadRequestAlertException("Artista no encontrado", "album", "artistnotfound"));

album.setArtist(artist);
album.setActive(false); // Los álbumes nuevos se crean inactivos hasta que el admin los active
album = albumRepository.save(album);

return albumMapper.toDto(album);
}
```

El detalle de `album.setActive(false)` garantiza que ningún álbum recién creado sea visible públicamente sin una revisión previa por parte de un administrador o editor. Este flujo de moderación protege el catálogo de contenido inapropiado o incompleto.

`BadRequestAlertException` extiende `RuntimeException` y es capturada por `ExceptionHandler` (`@ControllerAdvice`), que la transforma en una respuesta JSON con el estándar RFC 7807 Problem Details:

```
{
  "type": "https://www.jhipster.tech/problem/constraint-violation",
  "title": "Bad Request",
  "status": 400,
  "detail": "Artista no encontrado",
  "entityName": "album",
  "errorKey": "artistnotfound"
}
```

3.10.3 Asignación de usuario propietario en PlaylistServiceImpl

Las playlists no están vinculadas a un perfil de artista, sino directamente a cualquier usuario autenticado.

`PlaylistServiceImpl` usa `SecurityContextHolder` para obtener la identidad del usuario actual:

```
private User getCurrentUser() {
    Authentication authentication = SecurityContextHolder.getContext().getAuthentication();
    String login = authentication.getName(); // nombre del principal autenticado
    return userRepository.findOneByLogin(login)
        .orElseThrow(() -> new RuntimeException("Usuario no encontrado"));
}
```

```

    }

    @Override
    public PlaylistDTO save(PlaylistDTO playlistDTO) {
        Playlist playlist = playlistMapper.toEntity(playlistDTO);
        User user = getCurrentUser();
        playlist.setUser(user);
        playlist = playlistRepository.save(playlist);
        return playlistMapper.toDto(playlist);
    }

    @Override
    public PlaylistDTO update(PlaylistDTO playlistDTO) {
        Playlist playlist = playlistMapper.toEntity(playlistDTO);
        User user = getCurrentUser();
        playlist.setUser(user); // Re-asignar en cada actualización
        playlist = playlistRepository.save(playlist);
        return playlistMapper.toDto(playlist);
    }
}

```

El método `getCurrentUser()` se extrae como método privado reutilizable para evitar duplicación entre `save()` y `update()`. La re-asignación del usuario en `update()` previene que una petición maliciosa pueda transferir la propiedad de una playlist a otro usuario mediante manipulación del DTO.

3.10.4 Adición de canciones a una playlist

El método `addSongToPlaylist` implementa lógica adicional: verifica que la relación no exista antes de crearla, evitando duplicados en la tabla `playlist_song`:

```

@Override
public void addSongToPlaylist(Long playlistId, Long songId) {
    Playlist playlist = playlistRepository.findById(playlistId)
        .orElseThrow(() -> new RuntimeException("Playlist no encontrada"));
    Song song = songRepository.findById(songId)
        .orElseThrow(() -> new RuntimeException("Canción no encontrada"));

    // Comprobar si la canción ya está en la playlist
    boolean exists = playlistSongRepository
        .findByPlaylistIdAndSongId(playlistId, songId).isPresent();
    if (exists) return; // Idempotente: no lanza error si ya existe

    PlaylistSong ps = new PlaylistSong();
}

```

```
ps.setPlaylist(playlist);  
ps.setSong(song);  
ps.setAddedAt(Instant.now()); // Marca temporal de adición  
playlistSongRepository.save(ps);  
}
```

La operación es **idempotente**: si la canción ya está en la playlist, el método retorna silenciosamente sin lanzar error ni crear duplicados. Este diseño simplifica el cliente, que puede llamar al endpoint varias veces sin efectos colaterales.

3.11 Sistema de subida y streaming de ficheros

El controlador `FileUploadResource` (`@RestController @RequestMapping("/api/upload")`) centraliza todas las operaciones de subida y entrega de ficheros. Se diseñó para separar completamente la gestión de ficheros binarios de los controladores de entidades, manteniendo limpia la API REST principal.

3.11.1 Subida de imágenes de portada (POST /api/upload/image)

```
POST /api/upload/image  
Content-Type: multipart/form-data  
Authorization: Bearer <token>  
  
file: [fichero binario]
```

Proceso interno:

1. **Validación de tipo:** se verifica que `Content-Type` comience por `image/` . Si no es una imagen, se devuelve HTTP 400 con `{"error": "Solo se permiten imágenes"}` .
2. **Creación del directorio:** `Files.createDirectories(uploadPath)` crea el directorio `uploads/` si no existe, de forma recursiva.
3. **Nombre único:** se genera `UUID.randomUUID().toString() + extensión` , donde la extensión se extrae del nombre de fichero original (`originalFilename.lastIndexOf(".")`). El UUID garantiza que dos subidas simultáneas del mismo fichero nunca colisionen.
4. **Escritura a disco:** `Files.copy(file.getInputStream(), filePath, StandardCopyOption.REPLACE_EXISTING)` transfiere el contenido en streaming, sin cargar el fichero entero en memoria.
5. **Respuesta:** `{"url": "/uploads/{uuid}.{ext}"}` . El frontend almacena esta URL en el campo `coverImage` de la entidad correspondiente (Song, Album, Artist).

```

@PostMapping("/image")
public ResponseEntity<Map<String, String>> uploadImage(@RequestParam("file") MultipartFile file) {
    String contentType = file.getContentType();
    if (contentType == null || !contentType.startsWith("image/")) {
        return ResponseEntity.badRequest().body(Map.of("error", "Solo se permiten imágenes"));
    }

    Path uploadPath = Path.of(uploadDir);
    if (!Files.exists(uploadPath)) Files.createDirectories(uploadPath);

    String extension = file.getOriginalFilename() != null
        ? file.getOriginalFilename().substring(file.getOriginalFilename().lastIndexOf("."))
        : ".jpg";

    String filename = UUID.randomUUID().toString() + extension;
    Files.copy(file.getInputStream(), uploadPath.resolve(filename), StandardCopyOption.REPLACE_EXISTING);

    return ResponseEntity.ok(Map.of("url", "/uploads/" + filename));
}

```

El límite de tamaño máximo por fichero se configura en `application.yml`:

```

spring:
  servlet:
    multipart:
      max-file-size: 15MB
      max-request-size: 15MB

```

Si el cliente supera este límite, Spring Boot rechaza la petición antes de llegar al controlador con HTTP 413

`Payload Too Large`.

3.11.2 Subida de ficheros de audio (POST /api/upload/audio)

El endpoint de audio sigue el mismo patrón que el de imagen, pero valida `contentType.startsWith("audio/")` y devuelve adicionalmente el campo `filename` para permitir referencias directas al fichero:

```

{"url": "/uploads/{uuid}.mp3", "filename": "{uuid}.mp3"}

```

La extensión por defecto cuando el nombre original no está disponible es `.mp3`. El frontend almacena la URL devuelta en el campo `fileUrl` de la entidad `Song`.

3.11.3 Streaming de audio con soporte de rangos HTTP (GET /api/upload/stream/{filename})

El endpoint de streaming permite reproducción directa desde el navegador con soporte de búsqueda en la línea de tiempo (scrubbing). Se devuelve la cabecera `Accept-Ranges: bytes`, que informa al cliente de que el servidor acepta peticiones de rango parcial (RFC 7233):

```
@GetMapping("/stream/{filename}")
public ResponseEntity<Resource> streamAudio(@PathVariable String filename) throws IOException {
    Path uploadPath = Path.of(uploadDir).toAbsolutePath().normalize();
    Path filePath = uploadPath.resolve(filename).normalize();

    // Protección contra path traversal: el fichero resuelto debe estar dentro del directorio de uplo
    if (!filePath.startsWith(uploadPath)) {
        return ResponseEntity.badRequest().build();
    }

    Resource resource = new UrlResource(filePath.toUri());
    if (!resource.exists() || !resource.isReadable()) {
        return ResponseEntity.notFound().build();
    }

    String contentType = Files.probeContentType(filePath);
    if (contentType == null) contentType = "audio/mpeg"; // Fallback para MP3

    return ResponseEntity.ok()
        .contentType(MediaType.parseMediaType(contentType))
        .header(HttpHeaders.CONTENT_DISPOSITION, "inline; filename=\"" + filename + "\"")
        .header(HttpHeaders.ACCEPT_RANGES, "bytes")
        .body(resource);
}
```

La protección contra **path traversal** es crítica: sin ella, un atacante podría solicitar

`/api/upload/stream/../../../../application.yml` para leer ficheros de configuración del servidor. La comprobación `filePath.startsWith(uploadPath)` garantiza que la ruta resuelta esté contenida dentro del directorio `uploads/`.

`Files.probeContentType(filePath)` detecta el tipo MIME real del fichero examinando su contenido (no solo la extensión), evitando que se sirvan ficheros con extensión incorrecta con un Content-Type equivocado.

La cabecera `Content-Disposition: inline` instruye al navegador para renderizar el audio en el reproductor HTML5 en lugar de descargarlo.

3.11.4 Integración frontend ↔ backend de subida

El formulario de creación de canciones en Angular llama secuencialmente a los endpoints de subida antes de enviar el DTO de la canción:

1. POST /api/upload/image → {url: "/uploads/uuid.jpg"} → coverImageUrl
2. POST /api/upload/audio → {url: "/uploads/uuid.mp3"} → fileUrl
3. POST /api/songs {title, ..., coverImage: coverImageUrl, fileUrl: fileUrl}

Este diseño garantiza que los ficheros binarios y los metadatos de la canción se gestionan mediante flujos HTTP independientes, evitando peticiones multipart excesivamente grandes que incluyan tanto el JSON como los ficheros.

3.12 Repositorios personalizados y consultas JPQL

Spring Data JPA genera automáticamente las operaciones CRUD básicas a partir de las interfaces de repositorio. Sin embargo, varios requisitos del dominio requieren consultas específicas que se implementan mediante anotaciones `@Query` con JPQL (Java Persistence Query Language).

ArtistRepository

```
@Repository
public interface ArtistRepository extends JpaRepository<Artist, Long> {

    // Buscar el perfil de artista vinculado a un login de usuario
    @Query("SELECT a FROM Artist a WHERE a.user.login = :login")
    Optional<Artist> findByUserLogin(@Param("login") String login);

    // Búsqueda de artistas por nombre (insensible a mayúsculas, paginado)
    Page<Artist> findByNameContainingIgnoreCase(String name, Pageable pageable);
}
```

La primera consulta es fundamental para el patrón de ownership: dado el login del usuario autenticado, obtiene el `Artist` correspondiente para asignarlo automáticamente a canciones y álbumes. La relación entre `User` y `Artist` es `@OneToOne` bidireccional; la consulta JPQL navega por ella mediante notación de punto (`a.user.login`).

SongRepository (métodos clave)

```

@Repository
public interface SongRepository extends JpaRepository<Song, Long>,
    SongRepositoryWithBagRelationships {

    // Canciones de un artista específico (para "mis canciones")
    Page<Song> findByArtistLogin(String login, Pageable pageable);

    // Búsqueda en tiempo real por título
    Page<Song> findByTitleContainingIgnoreCaseAndActiveTrue(String title, Pageable pageable);

    // Canciones públicas (activas y con fecha de lanzamiento <= hoy)
    @Query("SELECT s FROM Song s WHERE s.active = true AND s.releaseDate <= :today")
    List<Song> findPublicSongs(@Param("today") LocalDate today);

    // Canciones públicas de un álbum concreto
    @Query("SELECT s FROM Song s WHERE s.album.id = :albumId AND s.active = true AND s.releaseDate <= :today")
    List<Song> findPublicSongsByAlbumId(@Param("albumId") Long albumId, @Param("today") LocalDate today);

    // Canciones activas (paginadas)
    Page<Song> findByActiveTrue(Pageable pageable);
}

```

El método `findByTitleContainingIgnoreCaseAndActiveTrue` es generado automáticamente por Spring Data JPA a partir del nombre del método mediante su convención de nomenclatura: `ContainingIgnoreCase` genera un `LIKE '%?%'` en SQL insensible a mayúsculas, y `AndActiveTrue` añade `AND active = 1`. Esto proporciona búsqueda en tiempo real sin escribir SQL manual.

PlaylistRepository

```

@Repository
public interface PlaylistRepository extends JpaRepository<Playlist, Long> {

    // Playlists de un usuario (para "mis playlists")
    List<Playlist> findByUserLogin(String login);

    // Playlists públicas
    List<Playlist> findByIsPublicTrue();
}

```


`findByUserLogin` navega la relación `@ManyToOne` de `Playlist` con `User` para filtrar por el login del propietario. Spring Data JPA genera la consulta JPQL `SELECT p FROM Playlist p WHERE p.user.login = :login` automáticamente.

SongRepositoryWithBagRelationships

La interfaz `SongRepositoryWithBagRelationships` resuelve el problema del producto cartesiano de Hibernate al cargar relaciones `ManyToMany` en colecciones de tipo *bag* (no ordenadas). Se implementa en

`SongRepositoryWithBagRelationshipsImpl`:

```
// Cargar todas las canciones y luego los artistas en una segunda consulta
@Override
public Page<Song> findAllWithEagerRelationships(Pageable pageable) {
    List<Long> ids = songRepository.findAllIds(pageable); // 1ª consulta: IDs
    List<Song> songs = songRepository.findSongsByIds(ids); // 2ª consulta: entidades + artistas
    return PageImpl<>(songs, pageable, songRepository.count());
}
```

Separar en dos consultas evita que Hibernate genere un `CROSS JOIN` que multiplica las filas (una por cada artista asociado), lo que habría producido resultados duplicados en la paginación.

3.13 Servicio de búsqueda y filtrado

La búsqueda de canciones por título se implementa en `SongServiceImpl` mediante el repositorio personalizado:

```
@Override
public Page<SongDTO> findByTitleContaining(String title, Pageable pageable) {
    return songRepository
        .findByTitleContainingIgnoreCaseAndActiveTrue(title, pageable)
        .map(songMapper::toDto);
}
```

El controlador `SongResource` expone este método como un endpoint GET con parámetro opcional:

```
GET /api/songs/search?title=bohemian&page=0&size=10
```

En el frontend, el componente de búsqueda usa `debounceTime(300)` de RxJS para evitar peticiones en cada pulsación de tecla: la búsqueda se lanza solo 300 ms después de que el usuario deja de escribir. Esta técnica reduce significativamente la carga del backend sin perjudicar la experiencia de usuario.

3.14 Control de acceso por rol en endpoints REST

Los controladores de recursos aplican restricciones de acceso mediante `@PreAuthorize` a nivel de método, combinando granularidad fina con declaratividad. Ejemplos representativos:

```
// AlbumResource: solo ROLE_ADMIN y ROLE_ARTIST pueden crear álbumes
@PostMapping("")
@PreAuthorize("hasAnyAuthority('ROLE_ADMIN', 'ROLE_ARTIST')")
public ResponseEntity<AlbumDTO> createAlbum(@Valid @RequestBody AlbumDTO albumDTO) { ... }

// AlbumResource: solo ROLE_ADMIN puede eliminar álbumes
@DeleteMapping("/{id}")
@PreAuthorize("hasAuthority('ROLE_ADMIN')")
public ResponseEntity<Void> deleteAlbum(@PathVariable Long id) { ... }

// GenreResource: CRUD de géneros solo para administradores
@PostMapping("")
@PreAuthorize("hasAuthority('ROLE_ADMIN')")
public ResponseEntity<GenreDTO> createGenre(@Valid @RequestBody GenreDTO genreDTO) { ... }
```

La anotación `@PreAuthorize` es evaluada por Spring Security AOP antes de ejecutar el método. Si el usuario no tiene el rol requerido, Spring Security lanza `AccessDeniedException`, que `ExceptionHandler` convierte en HTTP 403 Forbidden.

Esta estrategia es preferible a gestionar la autorización dentro de la lógica del controlador porque:

- Es declarativa y legible: el rol requerido es visible sin leer el cuerpo del método.
- Es consistente: Spring Security aplica la misma política tanto a peticiones HTTP como a llamadas internas entre servicios.
- Reduce el riesgo de errores de omisión: si se añade un nuevo endpoint, el desarrollador recibe un acceso denegado por defecto hasta que configure el permiso explícitamente.

4. Tecnologías del frontend

4.1 Angular 21

Se eligió Angular 21 por:

- **Tipado estático fuerte** con TypeScript, que reduce errores en tiempo de ejecución.
- **Arquitectura basada en componentes standalone**, que simplifica la estructura del proyecto eliminando la necesidad de módulos NgModule.

- **Sistema de señales reactivas** (`signal` , `computed`) para la gestión de estado local sin necesidad de librerías externas.
- **CLI potente** que facilita la generación de código, construcción y pruebas.

4.2 Bootstrap 5 + SCSS

Se empleó Bootstrap 5 para la maquetación responsive y los componentes UI base. Las personalizaciones visuales se realizaron mediante variables SCSS, lo que permite modificar el tema global desde un único punto sin alterar el código de Bootstrap.

4.3 Font Awesome 7

Iconografía vectorial escalable integrada mediante `@fortawesome/angular-fontawesome` . Los iconos se registran globalmente en `app.ts` mediante la librería de iconos, evitando importaciones duplicadas.

4.4 RxJS 7

Utilizado para la gestión de flujos asíncronos (peticiones HTTP, eventos de estado). Se usa principalmente en los servicios de entidades para transformar y manejar respuestas del backend.

4.5 ngx-translate

Internacionalización (i18n) del frontend con soporte para español e inglés. Las traducciones se definen en archivos JSON bajo `src/main/webapp/i18n/` .

5. Diseño UI/UX

5.1 Tema oscuro estilo Spotify

La decisión de adoptar un diseño oscuro similar a Spotify responde a criterios de UX específicos para aplicaciones de música:

- Los fondos oscuros reducen la fatiga visual en sesiones largas de escucha.
- El contraste con portadas de álbumes (generalmente coloridas) destaca el contenido multimedia.
- La familiaridad del patrón de diseño reduce la curva de aprendizaje para el usuario.

Paleta principal:

Token	Color	Uso
<code>\$bg-primary</code>	<code>#0f172a</code>	Fondo general
<code>\$bg-secondary</code>	<code>#1e293b</code>	Cards, sidebar

Token	Color	Uso
<code>\$accent</code>	<code>#38bdf8</code>	Elementos activos, hover
<code>\$text-primary</code>	<code>#ffffff</code>	Texto principal
<code>\$text-secondary</code>	<code>#b3b3b3</code>	Texto secundario

5.2 Dashboards diferenciados por rol

Se implementaron tres dashboards distintos que se muestran automáticamente según el rol del usuario autenticado:

Rol	Dashboard	Contenido principal
<code>ROLE_ADMIN</code>	<code>dashboard-admin</code>	Gestión de usuarios, métricas del sistema, accesos directos a entidades
<code>ROLE_EDITOR</code> / <code>ROLE_ARTIST</code>	<code>dashboard-editor</code>	Catálogo musical, gestión de álbumes y canciones propias
<code>ROLE_USER</code>	<code>dashboard-user</code>	Listas de reproducción, canciones favoritas, historial

La redirección al dashboard correcto se realiza en el componente `HomeComponent` mediante comprobación de roles con el servicio `AccountService`.

5.3 Barra lateral (Sidebar)

La sidebar ofrece navegación rápida y varía su contenido según el rol. Incluye:

- Enlace al dashboard de inicio.
- Accesos a entidades relevantes para el rol.
- Estado contraído/expandido gestionado por `SidebarService`.

5.4 Barra del reproductor (Player Bar)

Barra de reproducción fija en la parte inferior de la pantalla, siempre visible. Controles: reproducir/pausar, anterior, siguiente, aleatorio, repetir, volumen y mute. Implementada como componente standalone con señales Angular para el estado interno.

5.5 Panel de letras

Integrado en la barra del reproductor. Al pulsar el botón de letras, se despliega un panel encima de la barra que muestra la letra de la canción en reproducción, obtenida en tiempo real desde la API `lyrics.ovh`. Incluye indicador de carga y mensaje de error si no se encuentran letras.

6. Seguridad del frontend

6.1 Autenticación JWT

Flujo de autenticación:

1. El usuario envía credenciales al endpoint `/api/authenticate`.
2. El backend valida y devuelve un token JWT.
3. El frontend almacena el token y lo incluye en todas las peticiones mediante el interceptor `authInterceptor`.
4. Al expirar el token, `authExpiredInterceptor` redirige automáticamente a la página de login.

6.2 Guards de rutas

Rutas protegidas con `AuthGuard` que verifica la existencia y validez del token antes de permitir la navegación. Las rutas de administración requieren el rol `ROLE_ADMIN`.

6.3 Directivas de autorización por rol

Los elementos de la UI que solo deben mostrarse a ciertos roles se controlan mediante comprobaciones en los componentes con el método `hasAnyAuthority()` del servicio `AccountService`.

7. Integración de la API de letras

7.1 API seleccionada: lyrics.ovh

Se eligió `lyrics.ovh` por:

- **Gratuita y sin clave de API:** no requiere registro ni límites de uso que afecten al TFG.
- **CORS habilitado:** permite peticiones directas desde el navegador sin necesidad de un proxy en el backend.
- **Simple:** endpoint único `GET https://api.lyrics.ovh/v1/{artista}/{titulo}`.

7.2 Implementación

El servicio `LyricsService` encapsula la petición HTTP externa. El componente de la barra del reproductor inyecta este servicio y gestiona los estados de carga, éxito y error mediante señales Angular. El panel se muestra/oculta con un botón en el área derecha de la barra del reproductor.

8. Problemas encontrados y soluciones

8.1 Integración de roles personalizados

JHipster genera por defecto los roles `ROLE_USER` y `ROLE_ADMIN`. Para añadir `ROLE_EDITOR` y `ROLE_ARTIST` fue necesario:

- Añadir los nuevos roles en la tabla `jhi_authority` mediante una migración Liquibase dedicada (`20260501210000_add_editor_artist_authorities.xml`).
- Registrarlos como constantes en `AuthoritiesConstants.java`.
- Adaptar los guards y las comprobaciones de roles en el frontend.

8.2 Diseño responsive de la sidebar y el player

La coexistencia de sidebar fija, navbar superior y player bar inferior requirió un sistema de márgenes y z-index coordinados en el SCSS global para evitar solapamientos en distintos tamaños de pantalla.

8.3 Estado del reproductor sin backend de audio

El reproductor visual está implementado pero la reproducción de audio real depende de que las canciones tengan una URL de archivo válida en el campo `fileUrl`. Para la demo se utilizan estados visuales con señales Angular.

8.4 Carga de relaciones ManyToMany en JPA

La relación `Song ↔ Artist` es ManyToMany. Hibernate genera un producto cartesiano si se usa una única consulta JPQL con `JOIN FETCH`. Se resolvió implementando `SongRepositoryWithBagRelationships`, que separa la carga en dos consultas distintas: una para las canciones y otra para sus artistas, evitando duplicados y mejorando el rendimiento.

8.5 Streaming de audio y soporte de rangos HTTP

La reproducción de audio en el navegador mediante el elemento `<audio>` de HTML5 requiere que el servidor soporte peticiones de rango parcial (cabecera `Range: bytes=X-Y`). Sin esta capacidad, el navegador no puede saltar a una posición arbitraria del fichero (scrubbing) sin descargar el contenido completo hasta ese punto.

El problema se detectó durante las pruebas de integración: al intentar avanzar en la canción desde el reproductor, el navegador enviaba peticiones de rango que el servidor rechazaba por no incluir la cabecera `Accept-Ranges: bytes` en la respuesta.

La solución fue añadir `HttpHeaders.ACCEPT_RANGES` en la respuesta del endpoint `/api/upload/stream/{filename}` y retornar un `UrlResource` que Spring convierte en una respuesta parcial cuando recibe la cabecera `Range`. Spring Boot gestiona automáticamente las respuestas `206 Partial Content` cuando el cliente solicita un rango y el recurso está envuelto en una `UrlResource`.

8.6 Colisión de nombres de fichero en subidas concurrentes

En una primera versión del endpoint de subida se usaba el nombre original del fichero del cliente para guardar en disco. Esto generaba dos problemas:

1. **Colisión:** si dos usuarios subían un fichero con el mismo nombre (por ejemplo, `portada.jpg`), el segundo sobrescribía al primero.
2. **Caracteres peligrosos:** nombres con espacios, tildes o caracteres especiales podían causar problemas al construir la URL de descarga.

La solución fue generar un nombre completamente nuevo con `UUID.randomUUID().toString()` concatenado a la extensión original del fichero. Los UUIDs son identificadores de 128 bits generados de forma pseudoaleatoria con una probabilidad de colisión despreciable (1 en 2^{122}). La extensión se preserva para que el servidor pueda detectar el tipo MIME correctamente.

8.7 Vulnerabilidad de path traversal en el endpoint de streaming

Al implementar el endpoint `GET /api/upload/stream/{filename}`, se identificó un riesgo de seguridad potencial: si un atacante enviaba un nombre de fichero como `../config/application.yml`, la ruta resuelta podría salir del directorio `uploads/` y acceder a ficheros sensibles del servidor.

La mitigación se implementó en dos pasos:

```
Path uploadPath = Path.of(uploadDir).toAbsolutePath().normalize();
Path filePath = uploadPath.resolve(filename).normalize();

// Verificar que el fichero resuelto está dentro del directorio permitido
if (!filePath.startsWith(uploadPath)) {
    return ResponseEntity.badRequest().build();
}
```

`normalize()` elimina los segmentos `..` y `.` de la ruta antes de la comprobación. `toAbsolutePath()` convierte la ruta relativa a absoluta para que `startsWith()` funcione correctamente. Este es el patrón canónico recomendado por OWASP para prevenir path traversal en Java.

8.8 Orden de changelogs en master.xml de Liquibase

Durante el desarrollo en ramas paralelas, el fichero `master.xml` que orquesta la ejecución de los changelogs sufrió conflictos de merge. En un commit, los changelogs quedaron ordenados de forma incorrecta: `Like` antes que `Song`, cuando `Like` tiene una clave foránea hacia `Song`.

El resultado fue que Liquibase fallaba al arrancar el backend con un error de clave foránea referenciando una tabla inexistente. La solución requirió reordenar manualmente los includes en `master.xml` para respetar el orden de dependencias: primero las entidades sin dependencias externas (`Genre`, `Artist`), luego las que dependen de ellas (`Album`, `Song`), y finalmente las tablas de relación (`PlaylistSong`, `Like`, `Play`).

8.9 Configuración del secreto JWT mediante variable de entorno

La configuración inicial de JHipster incluía el secreto JWT directamente en `application.yml` como una cadena Base64 hardcodeada:

```
jhipster:
  security:
    authentication:
      jwt:
        base64-secret: "mi-secreto-hardcodeado-base64..."
```

Incluir secretos criptográficos en el repositorio de control de versiones es una vulnerabilidad grave: cualquier persona con acceso al repositorio puede obtener el secreto y forjar tokens JWT arbitrarios.

La solución fue reemplazar el valor por una referencia a variable de entorno:

```
jhipster:
  security:
    authentication:
      jwt:
        base64-secret: "${JWT_SECRET}"
```

En el entorno de desarrollo, `JWT_SECRET` se define en un fichero `.env` local (excluido del repositorio mediante `.gitignore`). En producción, se configura como variable de entorno del servidor o en el sistema de secrets del orquestador de contenedores.

8.10 Endpoint raíz GET /api/albums retornaba 405 Method Not Allowed

Durante la integración frontend-backend, el componente de álbumes realizaba una petición `GET /api/albums` al cargar la lista. El backend retornaba 405 `Method Not Allowed` porque `AlbumResource` no declaraba un método `@GetMapping("")` en la ruta raíz: solo tenía rutas con parámetros (`/ {id}`) o con `@RequestMapping` de subpath.

La solución fue añadir el método explícito:

```
@GetMapping("")
@PermitAll
public ResponseEntity<List<AlbumDTO>> getAllAlbums(
    @org.springdoc.core.annotations.ParameterObject Pageable pageable) {
    Page<AlbumDTO> page;
    String login = SecurityUtils.getCurrentUserLogin().orElse(null);
    if (login != null && accountService.hasAnyAuthority(AuthoritiesConstants.ARTIST, AuthoritiesConst
```



```
        page = albumService.findAll(pageable);
    } else {
        page = albumService.findPublicAlbums(pageable);
    }

    HttpHeaders headers = PaginationUtil.generatePaginationHttpHeaders(
        ServletUriComponentsBuilder.fromCurrentRequest(), page);
    return ResponseEntity.ok().headers(headers).body(page.getContent());
}
```

La lógica de filtrado por rol en el propio endpoint permite que los usuarios anónimos y oyentes vean solo álbumes activos, mientras que artistas y administradores ven todo el catálogo.

9. Conclusiones y trabajo futuro

9.1 Conclusiones

El proyecto MusicPlayer ha permitido aplicar en un caso real los conocimientos adquiridos durante el grado, cubriendo las principales dimensiones del desarrollo de software moderno: arquitectura cliente-servidor, seguridad basada en tokens, diseño de interfaz de usuario orientado a la experiencia, persistencia relacional versionada y consumo de APIs externas.

Arquitectura y tecnología base

La elección de JHipster 9.0.0 como generador de código base fue acertada para el contexto del proyecto. En lugar de dedicar semanas a configurar Spring Security, Liquibase, MapStruct y la estructura de paquetes, el equipo dispuso de una base funcional desde el primer día, y pudo centrarse en las contribuciones de valor diferencial: el diseño visual, la lógica de propiedad, la subida de ficheros y las funcionalidades de usuario final.

La separación de responsabilidades en tres capas bien delimitadas (controlador REST, servicio, repositorio) facilitó el trabajo en paralelo: el desarrollador de frontend podía consumir los endpoints mientras el desarrollador de backend refinaba la lógica de negocio, con mínimas dependencias entre ramas de trabajo.

Frontend: Angular 21 con señales

La adopción de las señales reactivas (`signal` , `computed`) de Angular 16+ para la gestión de estado del reproductor demostró ser una elección sólida. Comparado con alternativas como NgRx (Redux pattern) o BehaviorSubjects de RxJS, las señales ofrecen:

- **Menor superficie de código:** el estado del reproductor se define en 8 líneas de `signal()` y `computed()` frente a las 4 clases (actions, reducer, effects, selectors) que requeriría NgRx.

- **Reactividad de grano fino:** Angular re-renderiza solo los elementos del template que leen una señal que cambió, sin necesidad de `ChangeDetectionStrategy.OnPush` manual.
- **Sin dependencias externas:** NgRx añade ~400KB gzipped al bundle; las señales son parte del core de Angular.

La arquitectura de componentes standalone eliminó los módulos NgModule, reduciendo el boilerplate y mejorando la legibilidad de las importaciones. El lazy loading por ruta se configuró directamente en el array de rutas.

Backend: patrón de propiedad y seguridad

La implementación del patrón de ownership en la capa de servicio (sección 3.10) resolvió un requisito de seguridad no trivial de forma elegante. Al extraer el usuario autenticado directamente del `SecurityContext` de Spring en los métodos de servicio, se garantiza que la identidad del propietario no puede ser suplantada mediante manipulación del cuerpo de la petición HTTP. Este patrón es preferible a validar la propiedad en el controlador porque mantiene la lógica de negocio separada de la capa de presentación.

La gestión de errores centralizada mediante `ExceptionHandler` y `BadRequestAlertException` proporcionó respuestas HTTP estructuradas y predecibles que el frontend puede manejar de forma uniforme, sin necesidad de lógica de parsing de errores específica por endpoint.

Subida y streaming de ficheros

El módulo `FileUploadResource` cubrió un caso de uso fundamental para la funcionalidad de la plataforma: la subida de portadas y canciones sin intermediarios. El diseño en tres endpoints separados (imagen, audio, streaming) mantuvo cada operación simple y testeable de forma independiente. La protección contra path traversal aplicada en el endpoint de streaming es un ejemplo práctico de cómo las consideraciones de seguridad deben ser parte del diseño inicial, no una corrección posterior.

Pruebas E2E con Playwright

La suite de pruebas E2E desarrollada con Playwright cubre los flujos críticos de usuario de forma automatizada y reproducible. A diferencia de las pruebas unitarias que verifican lógica aislada, las pruebas E2E validan que el sistema completo (frontend + backend + base de datos) funciona correctamente desde la perspectiva del usuario. La cobertura de flujos de ownership (intentar editar el recurso de otro usuario) es especialmente valiosa porque estos errores son difíciles de detectar con pruebas unitarias.

Trabajo en equipo y control de versiones

El trabajo en ramas paralelas (`RamaAlex-detalles` para frontend y `RamaFran-Flujo/Back` para backend) permitió un desarrollo independiente con merges periódicos de integración. Los principales puntos de fricción fueron los ficheros de configuración compartidos (`application.yml`, `master.xml` de Liquibase) y los componentes Angular que llaman a endpoints nuevos antes de que existan. Para proyectos futuros, un entorno

de integración continua (CI) que ejecute automáticamente el build completo en cada merge habría detectado estos conflictos de forma inmediata.

9.2 Trabajo futuro

Las siguientes líneas de trabajo ampliarían significativamente las capacidades del sistema en un hipotético paso a producción:

Infraestructura y almacenamiento

- **Almacenamiento persistente en la nube:** reemplazar el directorio `uploads/` local por Amazon S3, Google Cloud Storage o equivalente. El endpoint `/api/upload` solo requeriría cambiar la implementación de escritura/lectura; la API externa (URLs devueltas al frontend) no cambiaría.
- **CDN para assets estáticos:** servir portadas de álbumes e imágenes de artistas desde una red de distribución de contenido para reducir latencia global y carga del servidor backend.
- **Despliegue con Docker Compose en producción:** el proyecto ya incluye `docker-compose.yml` para el entorno de desarrollo; completar el archivo para producción con variables de entorno seguras, healthchecks y reinicio automático.
- **CI/CD con GitHub Actions:** pipeline que ejecute el build Maven, las pruebas JUnit, el build Angular y los tests Playwright en cada Pull Request, bloqueando merges que rompan la suite de pruebas.

Funcionalidades de usuario final

- **Reproducción de audio real:** el campo `fileUrl` de `Song` ya almacena la URL del fichero; conectar el `<audio>` del reproductor Angular a `GET /api/upload/stream/{filename}` completaría la reproducción sin cambios en el backend.
- **Letras sincronizadas (formato LRC):** en lugar de mostrar la letra como bloque de texto estático, resaltar el verso actual en sincronía con el tiempo de reproducción usando el formato LRC (timestamps por línea). La API `lrc.lib.net` proporciona letras en este formato para muchas canciones.
- **Sistema de recomendación:** el historial de reproducciones almacenado en la tabla `Play` proporciona datos suficientes para implementar un motor de recomendación colaborativo básico que sugiera canciones basándose en los patrones de escucha del usuario.
- **Modo offline con Service Worker:** registrar un Service Worker en Angular para cachear el catálogo y permitir reproducción sin conexión de las canciones previamente descargadas.
- **Notificaciones en tiempo real:** integrar WebSocket (STOMP sobre SockJS, ya soportado por JHipster) para notificar al usuario cuando un artista que sigue publica contenido nuevo.

Calidad y mantenimiento

- **Pruebas unitarias Jest para servicios Angular:** los servicios críticos (`LyricsService`, `PlayerService`, `AccountService`) merecen una suite de pruebas unitarias que verifique su comportamiento con mocks de `HttpClient`, reduciendo la dependencia de pruebas E2E para validar lógica de frontend.

- **Pruebas de integración con Testcontainers:** reemplazar la base de datos H2 en memoria de las pruebas de integración del backend por una instancia real de MySQL levantada por Testcontainers. Esto garantiza que las consultas JPQL, los tipos de datos y los índices se comportan exactamente igual que en producción.
- **Análisis estático de seguridad:** integrar herramientas como OWASP Dependency-Check (para detectar dependencias con vulnerabilidades conocidas) y SonarQube (para análisis de código estático) en el pipeline de CI.
- **Documentación interactiva de la API:** generar automáticamente la documentación OpenAPI 3 con SpringDoc y exponerla en `/swagger-ui.html` para facilitar el consumo de la API por terceros o por el equipo de frontend sin necesidad de consultar el código fuente.

10. Glosario de términos técnicos

Término	Definición
JWT (JSON Web Token)	Estándar abierto (RFC 7519) para transmitir información de forma compacta y verificable entre partes mediante una firma digital.
JHipster	Generador de código que crea aplicaciones Spring Boot + Angular/React/Vue con configuración de seguridad, testing y CI preintegrada.
Liquibase	Herramienta de migración de esquemas de base de datos basada en changelogs versionados; garantiza reproducibilidad entre entornos.
MapStruct	Procesador de anotaciones Java que genera código de conversión entre entidades JPA y DTOs en tiempo de compilación.
DTO (Data Transfer Object)	Objeto plano que encapsula datos para transferirlos entre capas, desacoplando el modelo de dominio de la API REST.
JPQL	Java Persistence Query Language; lenguaje de consulta orientado a objetos para JPA, independiente del motor de base de datos.
UUID	Identificador Único Universal de 128 bits; probabilidad de colisión despreciable en la práctica (~1 en 2^{122}).
CORS	Cross-Origin Resource Sharing; mecanismo HTTP que controla qué orígenes pueden acceder a los recursos de un servidor.
Signal (Angular)	Primitiva reactiva de Angular 16+ que notifica automáticamente al framework de cambios de estado, habilitando re-renderizado de grano fino.
Path traversal	Vulnerabilidad de seguridad donde un atacante usa secuencias <code>../</code> para acceder a archivos fuera del directorio permitido.
RFC 7233	Estándar HTTP que define el mecanismo de peticiones de rango parcial (<code>Range: bytes=X-Y</code>), usado para streaming de audio/vídeo.

Término	Definición
HikariCP	Pool de conexiones JDBC de alto rendimiento; gestiona un conjunto de conexiones abiertas a la base de datos para reutilizarlas.
AOP (Aspect-Oriented Programming)	Paradigma que permite separar preocupaciones transversales (logging, seguridad) del código de negocio mediante aspectos.
E2E (End-to-End)	Tipo de prueba que verifica un flujo completo del sistema desde la perspectiva del usuario final, incluyendo frontend, backend y base de datos.
Playwright	Framework de automatización de navegadores desarrollado por Microsoft; permite escribir tests E2E en TypeScript/JavaScript.
OWASP	Open Web Application Security Project; organización que publica guías y herramientas de seguridad web, incluyendo el Top 10 de vulnerabilidades.